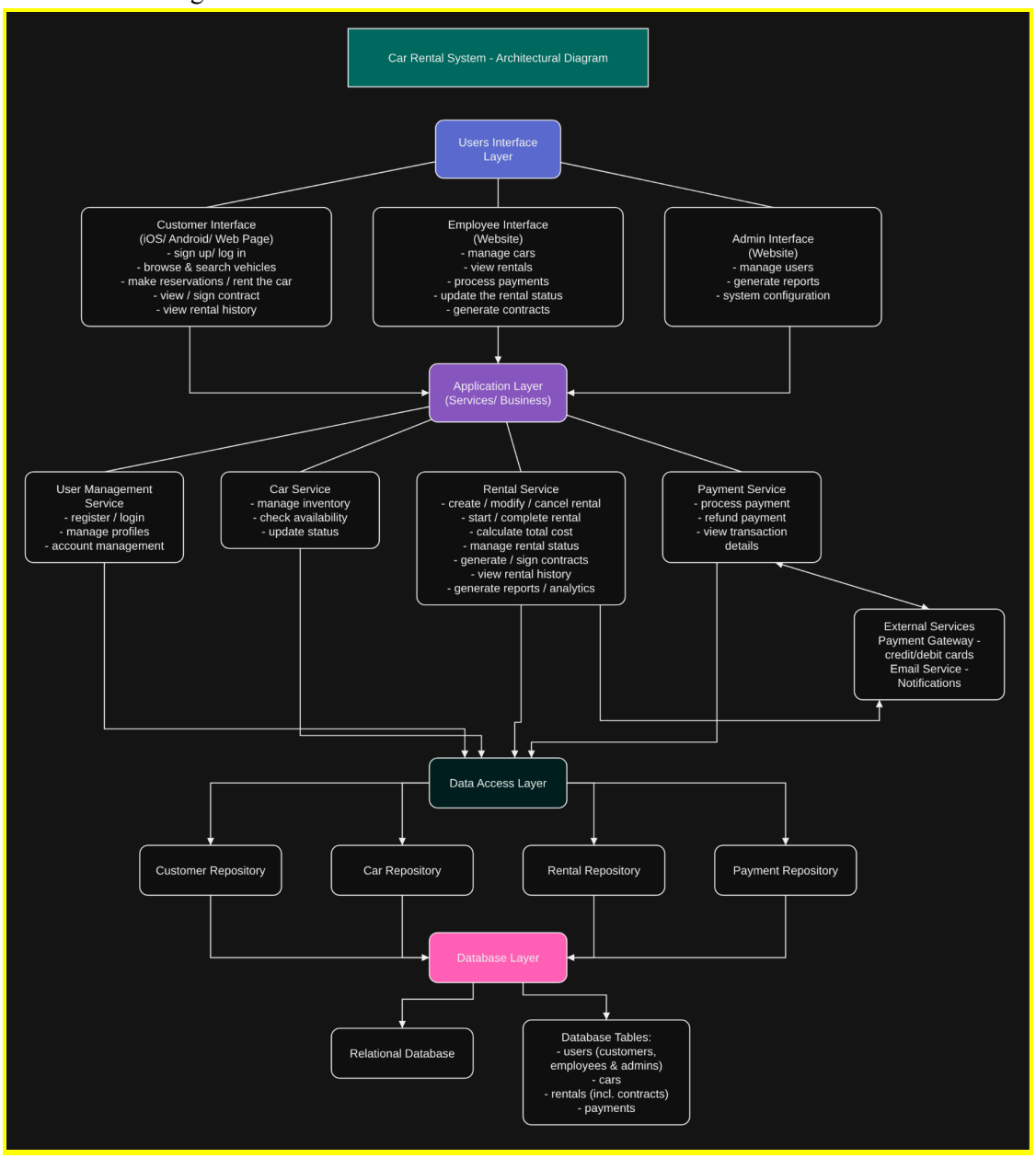
BeAvis Car Rental System

Chris E, Kevin P, Tigist T, Michelle V, Light O

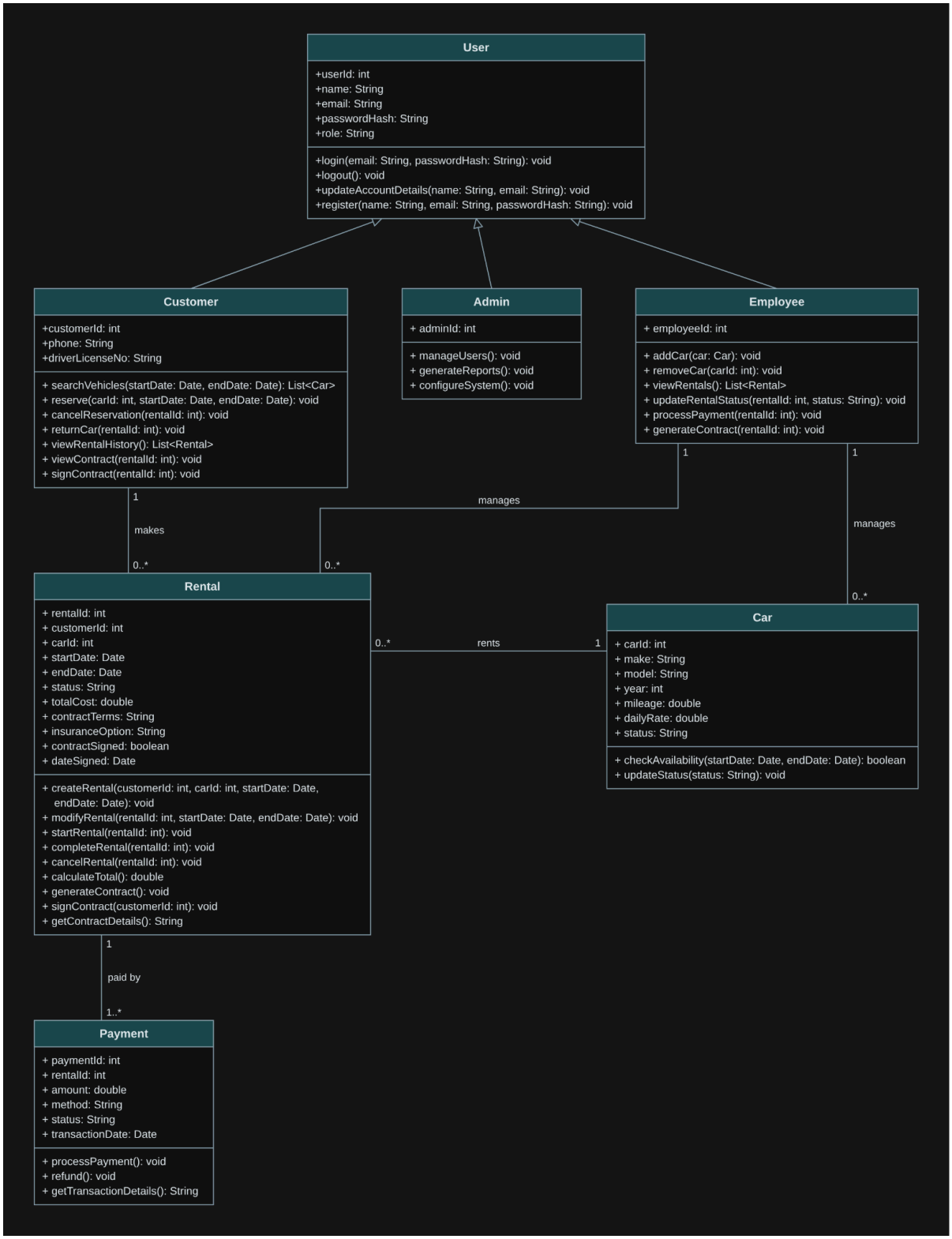
This is a car rental system designed to manage a car rental company. This system will allow customers to rent vehicles both online and via a mobile app, provided they have signed up for an account with the company. This system will also facilitate the distribution and signing of rental agreement contracts under secure conditions.

Employees will also be able to access the system in order to look up customers. Upon looking up a customer in the system, they will be able to see the rental status and contracts, check what cars are currently available, and update the status of cars.

Architectural Diagram:



UML Diagram:


User
+userId: int
+name: String
+email: String
+passwordHash: String
+role: String
+login(email: String, passwordHash: String): void
+logout(): void
+updateAccountDetails(name: String, email: String): void
+register(name: String, email: String, passwordHash: String): void
Customer
+customerId: int
+phone: String
+driverLicenseNo: String
+ searchVehicles(startDate: Date, endDate: Date): List<Car>
+ reserve(carId: int, startDate: Date, endDate: Date): void
+ cancelReservation(rentalId: int): void
+ returnCar(rentalId: int): void
+ viewRentalHistory(): List<Rental>
+ viewContract(rentalId: int): void
+ signContract(rentalId: int): void
Admin
+ adminId: int
+ manageUsers(): void
+ generateReports(): void
+ configureSystem(): void
Employee
+ employeeId: int
+ addCar(car: Car): void
+ removeCar(carId: int): void
+ viewRentals(): List<Rental>
+ updateRentalStatus(rentalId: int, status: String): void
+ processPayment(rentalId: int): void
+ generateContract(rentalId: int): void
1
makes
0..*
1
manages
0..*
1
manages
0..*
Rental
+ rentalId: int
+ customerId: int
+ carId: int
+ startDate: Date
+ endDate: Date
+ status: String
+ totalCost: double
+ contractTerms: String
+ insuranceOption: String
+ contractSigned: boolean
+ dateSigned: Date
+ createRental(customerId: int, carId: int, startDate: Date, endDate: Date): void
+ modifyRental(rentalId: int, startDate: Date, endDate: Date): void
+ startRental(rentalId: int): void
+ completeRental(rentalId: int): void
+ cancelRental(rentalId: int): void
+ calculateTotal(): double
+ generateContract(): void
+ signContract(customerId: int): void
+ getContractDetails(): String
0..*
rents
1
Car
+ carId: int
+ make: String
+ model: String
+ year: int
+ mileage: double
+ dailyRate: double
+ status: String
+ checkAvailability(startDate: Date, endDate: Date): boolean
+ updateStatus(status: String): void
1
paid by
1..*
Payment
+ paymentId: int
+ rentalId: int
+ amount: double
+ method: String
+ status: String
+ transactionDate: Date
+ processPayment(): void
+ refund(): void
+ getTransactionDetails(): String

User Class

Description

The User class represents a general person who uses the BeAvis system. It is the parent class of the Customer, Employee and Admin classes. It stores common account information and lets users create accounts, log in, log out and update their personal details. Passwords are never stored as plain text; the system stores a secure password hash instead. The role attribute records whether the user is a customer, employee or admin.

Attributes

- -userId: int
- -name: String
- -email: String
- -passwordHash: String
- -role: String

Operations

- +login(email: String, passwordHash: String): void
- +logout(): void
- +updateAccountDetails(name: String, email: String): void
- +register(name: String, email: String, passwordHash: String): void

Customer Class

Description

The Customer class represents a BeAvis customer. It inherits the common account information and operations from the User class. Each customer is identified by a customer ID and has contact and driver's license information on file. Customers can search for available cars, reserve a car, cancel a reservation, return a car and view their rental history. They can also view and sign the rental contract attached to each rental.

Attributes

- -customerId: int
- -phone: String
- -driverLicenseNo: String

Operations

- +searchVehicles(startDate: Date, endDate: Date): List<Car>
- +reserve(carId: int, startDate: Date, endDate: Date): void
- +cancelReservation(rentalId: int): void
- +returnCar(rentalId: int): void
- +viewRentalHistory(): List<Rental>

- +viewContract(rentalId: int): void
- +signContract(rentalId: int): void

Employee Class

Description
The Employee class represents a BeAvis employee. It inherits the common account information and operations from the User class. Employees have more permissions than customers because they manage the car inventory and customer rentals. They can add and remove cars, view rentals, update a rental's status, process payments and generate rental contracts when a customer picks up a car.

Attributes

- -employeeId: int

Operations

- +addCar(car: Car): void
- +removeCar(carId: int): void
- +viewRentals(): List<Rental>
- +updateRentalStatus(rentalId: int, status: String): void
- +processPayment(rentalId: int): void
- +generateContract(rentalId: int): void

Admin Class

Description
The Admin class represents a BeAvis system administrator. It inherits the common account information and operations from the User class. Admins have the highest level of access. They manage user accounts, generate business reports from rental and payment data, and configure system settings.

Attributes

- -adminId: int

Operations

- +manageUsers(): void
- +generateReports(): void
- +configureSystem(): void

Rental Class

Description

The Rental class represents a car rental made by a customer, from the initial reservation through the car's return. It stores the rental dates, status and total cost. The status shows whether the rental is Reserved, Active, Completed or Cancelled, so a reservation is simply a rental with the status "Reserved." Each rental is connected to one customer and one car, and can have one or more payments. The Rental class also holds the rental contract: its terms, insurance option, and whether and when the customer signed it. The class lets the system create, modify, start, complete and cancel rentals, calculate the total cost, and generate and sign contracts.

Attributes

- -rentalId: int
- -customerId: int
- -carId: int
- -startDate: Date
- -endDate: Date
- -status: String
- -totalCost: double
- -contractTerms: String
- -insuranceOption: String
- -contractSigned: boolean
- -dateSigned: Date

Operations

- +createRental(customerId: int, carId: int, startDate: Date, endDate: Date): void
- +modifyRental(rentalId: int, startDate: Date, endDate: Date): void
- +startRental(rentalId: int): void
- +completeRental(rentalId: int): void
- +cancelRental(rentalId: int): void
- +calculateTotal(): double
- +generateContract(): void
- +signContract(customerId: int): void
- +getContractDetails(): String

Car Class

Description

The Car class represents a vehicle in the BeAvis inventory. It stores the car's make, model, year, mileage, daily rental rate and current status. The status shows whether the car is available, reserved, rented or under maintenance. The daily rate is used to calculate a rental's total cost. Each car can be part of many rentals over time, but only one rental at a time.

Attributes

- -carId: int
- -make: String
- -model: String
- -year: int
- -mileage: double
- -dailyRate: double
- -status: String

Operations

- +checkAvailability(startDate: Date, endDate: Date): boolean
- +updateStatus(status: String): void

Payment Class

Description
The Payment class represents a payment made for a rental. It stores the amount, payment method, payment status and transaction date. Payments are processed through an external payment gateway that handles credit and debit cards. Each payment belongs to one rental, and a rental can have more than one payment, such as a deposit and a final charge. The class lets the system process payments, issue refunds and look up transaction details.

Attributes

- -paymentId: int
- -rentalId: int
- -amount: double
- -method: String
- -status: String
- -transactionDate: Date

Operations

- +processPayment(): void
- +refund(): void
- +getTransactionDetails(): String

Title page: Christopher
Software title: Christopher
Team members: Christopher
System Description: Christopher
Brief overview of system: Christopher
Software Architecture Overview
Architectural diagram of all major components: Michelle
UML Class Diagram: Kevin
Descriptions: Tigist
Development plan and timeline: Light
Partitioning of tasks: Light
Team member responsibilities: Light

Verification Test Plan: BeAvis Car Rental System

- ❖ Feature 1: Car Availability Tracking & Search

1. Unit Test

```
Car c
c.status = "AVAILABLE"

if (c.status == "AVAILABLE")
   return PASS
else
   return FAIL
```

Explanation: This unit test ensures that a Car object correctly holds and exposes its internal status attribute in memory.

2. Integration Test

```
Car c
c.carId = 42
c.status = "AVAILABLE"

Customer cust
cust.reserve()

if (c.status == "RESERVED")
   return PASS
else
   return FAIL
```

Explanation: This integration test verifies the interaction when a Customer executes the reserve() method. It ensures that placing a reservation successfully changes the linked Car object's status from "AVAILABLE" to "RESERVED" across the domain models.

3. System Test

Description: A customer logs into the BeAvis app and navigates to the "browse & search vehicles" page. They browse available vehicles in the inventory. Car 42 is currently flagged with status = "RESERVED" in the database, while Car 43 has status = "AVAILABLE". The system filters the inventory view so that Car 42 is hidden from the customer's browse list, while Car 43 is displayed as available for selection.

Explanation: This end-to-end system test verifies the vehicle search workflow across all layers. It ensures that the Customer Interface, Vehicle Service, Vehicle Repository, and Relational Database work together to restrict unavailable cars from appearing in customer search views.

❖ Feature 2: Rental Cancellation & Inventory Release

1. Unit Test

```
Rental r
r.rentalId = 101
r.status = "RESERVED"

r.cancelRental(101)

if (r.status == "CANCELLED")
   return PASS
else
   return FAIL
```

Explanation: This unit test checks the cancelRental(rentalId: int) method in the Rental class. It verifies that passing the target rentalId updates the object's status attribute from "RESERVED" to "CANCELLED".

2. Integration Test

```
Rental r
r.rentalId = 101
r.carId = 42

Car c
c.carId = 42
c.status = "RESERVED"

Customer cust
cust.cancelReservation(101)

if (c.status == "AVAILABLE")
   return PASS
else
   return FAIL
```

Explanation: This integration test checks how Customer.cancelReservation(rentalId: int) interacts with Rental and Car objects. It verifies that when a customer cancels a reservation, the system updates the linked Car object's status back to "AVAILABLE" so it can be booked again.

3. System Test

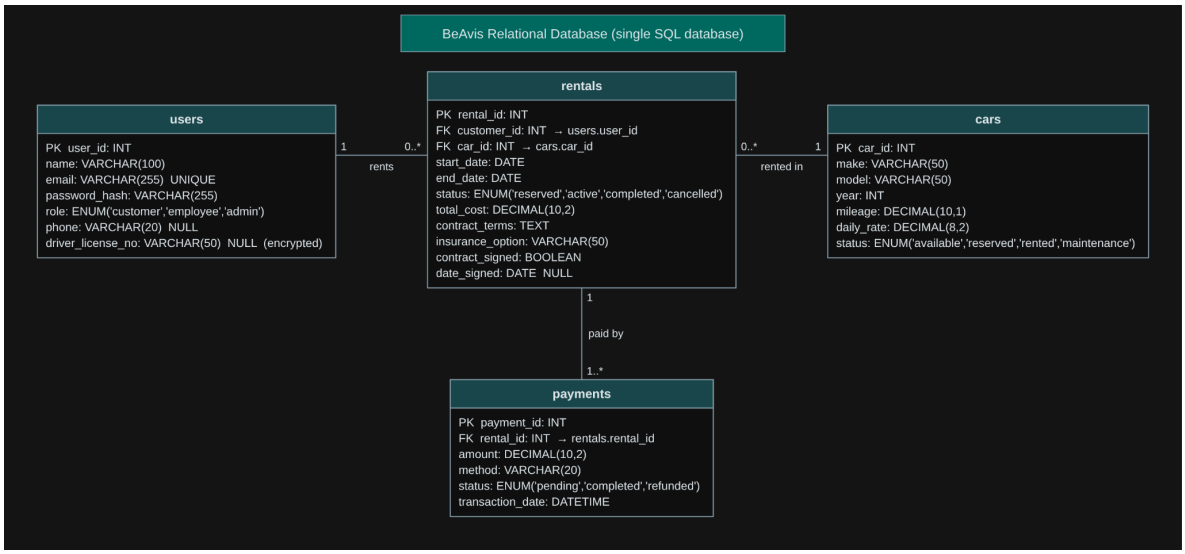
Description: A customer logs into their account on the BeAvis app and views their active rental history. They select rental #101 and click "Cancel Reservation" (triggering cancelReservation(101)). The system processes the request, updates the rental record to 'CANCELLED' in the rentals database table, restores car #42 status back to 'AVAILABLE' in the vehicles database table, and updates the screen with a cancellation success message.

Explanation: This system test verifies the cancellation flow across all system layers from UI user input, down to application services, repository updates, and database persistence.

Data Management Strategy:

What we're using

BeAvis uses one SQL database (like MySQL). All the data is stored in tables, and the tables are linked together. The app doesn't touch the database directly. It goes through the repositories (Customer, Car, Rental, and Payment Repository), as shown in our architecture diagram.



Why SQL?

- Every car, user, rental and payment has the same kinds of information, which fits neatly into tables.
- A customer makes rentals, a rental uses a car, and a rental gets paid for. SQL is built to link data like this.
- SQL keeps data safe from mistakes. For example, it can stop two customers from booking the same car at the same time.
- It's easy to make reports, like total money made or the most-rented cars.

Why only one database?

BeAvis is one company with a manageable amount of data. One database is simpler to build, back up and protect. Using more than one would add work without much benefit.

How we split up the data

We use 4 tables, one for each main class in our UML diagram:

users:	Customers, employees, and admins. A "role" column tells them apart.
cars:	Every car, its details, its daily price, and whether it's available
rentals:	Every reservation and rental, including the contract
payments:	Every payment and refund for a rental

- We put all users in one table because they all log in the same way and share most info.
- Reservations and contracts are stored inside the rentals table, because each rental only ever has one of each.
- Payments get their own table, because one rental can have more than one payment, such as a deposit and a final payment.

Keeping data safe

- Passwords are scrambled (hashed), never saved as plain text.
- We don't store card numbers. The outside payment service handles cards.
- Driver's license numbers are encrypted.
- Customers can only see their own information.
- The database is backed up every day.

Other options and tradeoffs

NoSQL (like MongoDB) instead of SQL

- Pros: more flexible, and easier to grow across many servers.
- Cons: worse at linking data together, and worse at preventing booking and payment mistakes.
- Why we didn't pick it: our data is organized and connected, so SQL fits better.

Different ways to organize the tables

- Separate tables for customers, employees and admins: cleaner, but we'd repeat a lot of the same info and login logic.
- Separate reservations and contracts tables: more detailed, but adds extra tables for things that always come with a rental anyway.

- A separate database just for payments: a bit more secure, but more complicated. Since we don't store card numbers, it isn't worth it.

What's changed?

Removed reservation and report services, including reports under the rental service, also subsequently removing the Reservation and Report Repositories. We also added contracts to the rental service and the rentals table in the SQL database. In the UML Class diagram we added the required classes, including an Admin class, and a payment class. Modified the methods within the classes to better match the class they are related to, including functions to manage contracts within the rental class.